

RP-06 — Authentication & Data Protection: Group Lesson Planner

Metadata

Field	Value
Report id	RP-06
Date	2026-08-20
Subject	Authentication approach and data-protection posture for a static, publicly-readable, single-user app whose maintainer and end user are different people
Inputs consumed	Research agent contract; verified-facts addendum; RP-01 §3 data model, §4 storage map, §8 defect list; RP-05 §2 candidate set, §6 recommendation, §7 implementation shape; <code>index.html</code> inspected at lines 278-300, 376-402, 1042-1053, 1086-1102; <code>.gitignore</code>
Statically read by me	<code>index.html</code> line boundaries of the default template block, confirmed by a redacting script that printed only line numbers, line lengths and a Cyrillic-present flag. Line 382 opens <code>defaultTemplate</code> , 386 is a heading, 387-392 carry the personal payment details , 395 is the Ukrainian payment-purpose line, 400 carries a personal first name , 401 closes the literal. This independently confirms RP-01 §8 D0 and RP-05 §8
Also statically read	Absence of any <code>placeholder</code> attribute anywhere in <code>index.html</code> (zero grep hits); the two <code>innerHTML</code> sinks at 1046-1051 and 1088-1101; the six-line <code>.gitignore</code>
Externally sourced	Every vendor claim and every legal claim in this report comes from the primary sources in §9, fetched 2026-08-20
Personal-data check	Verified by grep over this report: zero IBAN-shaped strings, zero Cyrillic characters, and the only run of eight or more digits anywhere is <code>20160504</code> inside the EUR-Lex CELEX identifier in source S21's URL, which is a document identifier and not personal data. The template values are referred to only by line reference and category, never reproduced
Not verified	Nothing was tested against a live Firebase project. No Firestore database was created, no security rule was deployed, no sign-in flow was exercised, and no authorized-domain entry was added. All authorization-model behaviour is documentation-sourced and labelled as such. No lawyer reviewed this report — see §10

Executive summary

The research pack's premise is wrong, and the correction changes the whole analysis. The pack assumed the app stores data about identified students. RP-01 §3 verified that it does not: there is no student entity, no attendance record, no payment record and no contact field. What is stored is teaching *groups* — a free-text group name, a per-lesson price, a currency, a list of lesson dates and per-month price overrides — plus one global currency setting and one global payment-message template. Whether

any personal data about a student exists at all therefore depends entirely on what the teacher types into one free-text field. `Tuesday intermediate` carries none. A child's full name carries a lot. Everything in §4 and §5 is built on that, not on the assumed model.

The one certain personal-data exposure runs the other way, and it is the report's central finding. A natural person's full name, personal bank IBAN and tax identification number are hardcoded in the default payment template at `index.html:387–392`, with a personal first name at `index.html:400`. This is published in the deployed page's view-source, is present in git history since the first commit (per the orchestrator's read, recorded in RP-01 §8 D0), and is copied into every generated payment message for any user who never edits the template. GitHub's own documentation states that Pages sites "are publicly available on the internet, even if the repository for the site is private" and that they "shouldn't be used for sensitive transactions like sending passwords or credit card numbers" [S18]. Sending your own bank details to someone who owes you money is normal; publishing them to the whole internet, alongside a tax identifier, is not. **The right posture is to treat those identifiers as permanently disclosed, not as recoverable by a commit.**

Six further conclusions:

1. **Authentication: agree with RP-05, on a stronger basis.** Google sign-in is right, but RP-05 argued it on session persistence and token models. The deciding axis is **recovery**, which RP-05 did not analyse. Every credential that lives only on the device — a passkey, an anonymous UID, a bookmarked unguessable URL — dies in exactly the event that started this programme. MDN states the case plainly for passkeys: "If a user loses an authenticator... they lose all the passkeys it contains" [S28]. Google sign-in is the only candidate whose recovery path survives a laptop wipe without the maintainer being involved.
2. **A new, concrete blocker RP-05 did not surface.** `signInWithRedirect` no longer works for apps not hosted on the Firebase auth domain unless one of Firebase's documented workarounds is implemented — required on "Google Chrome M115+... already required on Firefox 109+ and Safari 16.1+" [S8]. This app is hosted on `denyslystopadskyy.github.io`. The implementation must use `signInWithPopup`, and `denyslystopadskyy.github.io` must be added to Firebase's authorized-domain list, where "whitelisting a domain allows for requests from any URL and port of that domain" [S9] — so the grant covers every other GitHub Pages project the same account publishes.
3. **The "API keys are not secrets" claim is true but conditional, and the condition is the whole point.** Firebase's own wording is that keys *restricted to Firebase services* "do not need to be treated as secrets" **if** the app's setup follows the page's guidelines, and it names exceptions where a key must never be published [S1]. The security is done elsewhere: "Security of your Realtime Database, Cloud Firestore, and Cloud Storage data is enforced using Firebase Security Rules, and protection of covered APIs is by Firebase App Check — not by keeping your Firebase API key secret" [S1]. §2 gives the four-line rule and explains what each clause prevents.
4. **Nothing is hidden in a client bundle, and this repo does not even have a bundle.** Vite's own documentation says `VITE_*` variables "should not contain sensitive information such as API keys. The values of these variables are bundled into your source code at build time" [S17]. For this repo the point is more basic: there is no build step and no `package.json` on `main`, so the Firebase config is a literal in `index.html`. That is acceptable because the config is not a secret. **A static site has nowhere to put a secret.** That is an architectural boundary, not a configuration choice.

5. **GDPR: probably does not engage for the student-facing data today, and the household exemption is the weaker argument, not the strong one.** The strong argument is that there is no personal data about students unless she types one in. If she does, the exemption most likely fails: the CJEU requires it to be "narrowly construed", the word "purely" signals a restrictive scope, and it covers "only... activities which are carried out in the course of private or family life" [S22][S23]. Paid group teaching is an economic activity. Separately, Recital 18 puts controllers and processors that *provide the means* for personal or household processing inside the Regulation's scope [S20] — which is the maintainer, not the teacher. If the obligations do attach, they are light but real, and §7 lists them.
6. **The proportionate answer is roughly one paragraph of paperwork and four code fixes.** A compliance programme sized for a school district is the wrong answer, and so is pretending there are no obligations. §7 names both, and gives an explicit "not required here" list with a reason for each entry.

1. Authentication comparison table with recommendation

Six options, judged on the four axes the brief specifies. The user is one non-technical teacher; the maintainer is one part-time engineer. Nothing here assumes a second user ever exists.

1A — What it costs her, and what happens when she loses access

Option	End-user friction	Recovery after a device wipe	Revocable	Runs on GitHub Pages
Google sign-in (Firebase Auth)	One click, once. Session persists across browser restarts, per RP-05 §6	Google account recovery. Survives the device entirely; needs no maintainer involvement	Yes, per-account, by her or by Google	Yes, with <code>signInWithPopup</code> and an authorized-domain entry [S8] [S9]
Email link / OTP (Firebase email-link)	One inbox round-trip per new device or expired session	Her mailbox, which survives the device. Second-best recovery	Yes	Yes; same authorized-domain requirement
Shared passcode in the page	One string to remember or paste	Maintainer tells her the string again	No — rotation means a redeploy plus telling her	Yes, but see 1B
Device-bound key or passkey	Lowest — biometric or nothing	None. "If a user loses an authenticator... they lose all the passkeys it contains" [S28]	Yes, if the server keeps a registry	Yes
Unguessable identifier in the URL	None after the first bookmark	None if the bookmark went with the laptop	Only by migrating the data to a new id	Yes

Option	End-user friction	Recovery after a device wipe	Revocable	Runs on GitHub Pages
Firebase anonymous auth	None	None. The UID lives in origin storage and dies with the profile	n/a	Yes

1B — What it costs the maintainer, and what it actually buys

Option	Maintainer effort	Actual security value	Verdict
Google sign-in	Enable the provider, add one authorized domain, four rule lines [S7][S9]	Real. A per-user identity the rules can bind to, backed by Google's own account security	Recommended
Email link / OTP	Enable the provider; handle the same-device email match and store the address locally [S10]	Real, slightly weaker: possession of the mailbox is the whole credential	Documented fallback
Shared passcode	Trivial to write	Zero. It is in view-source of a public site [S18]; anyone who reads the page holds it	Reject
Device-bound key / passkey	Highest — a WebAuthn registration and recovery story, on a static host with no server	Strong against phishing (the credential is origin-scoped [S28]), useless against the failure mode that occurred	Reject
Unguessable identifier	Lowest	Weak. The URL leaks through history, browser sync, bookmark sharing and any screenshot; no revocation	Reject
Firebase anonymous auth	Lowest of the Firebase options	Nominal. Prevents world-writable data, provides no durable identity	Reject — agrees with RP-05 §7

Reconciliation with RP-05, and where this report adds to it

Agreed: Google sign-in only. No dissent. RP-05 reached that conclusion from session persistence and from Google Drive's browser token model. This report reaches the same conclusion from the recovery axis, which is the axis the incident actually tested, and the recovery axis is more decisive than either of RP-05's arguments.

Three things this report adds.

First, **recovery re-orders the field**. Ranked purely on "the laptop is gone and she opens the app on a borrowed machine", the order is: Google sign-in (Google's recovery) > email link (her mailbox) > everything else (nothing). The three device-local options — passkey, anonymous UID, bookmarked capability URL — are not merely weaker, they **reproduce the original incident**. A passkey is a good

credential attached to the wrong lifetime. RP-05 already ruled out anonymous auth for exactly this reason; the same reasoning kills the other two, and it should be stated once as a principle rather than three times as a coincidence.

Second, **signInWithRedirect is not available here**. Firebase documents that browser restrictions on cross-origin storage broke it, that a workaround became required on Chrome M115+ and was already required on Firefox 109+ and Safari 16.1+, and that apps not hosted on the Firebase Hosting `firebaseapp.com` subdomain must implement one of the listed options; `signInWithPopup` "avoids the cross-origin storage issue entirely" [S8]. Since RP-05 pins the app to GitHub Pages, **the implementation must use `signInWithPopup`**. One caveat so nobody reads that as unconditional: a popup has to be opened from a direct user gesture and can be blocked by a popup blocker, so the sign-in must hang off her click on a visible button and must degrade to a readable message rather than a silent no-op. Popup remains the only option that works off the Firebase auth domain without adding a custom domain or a reverse proxy [S8]. This is not a preference. It is the difference between a sign-in button that works and one that silently fails on the browsers she is most likely to be using.

Third, **the authorized-domain grant is coarser than it looks**. Firebase requires the app's host to be on the authorized-domain list; by default only `localhost` and the project's own hosting domain are on it, and "whitelisting a domain allows for requests from any URL and port of that domain" [S9]. Adding `denyslystopadskyy.github.io` therefore authorizes the OAuth flow for **every project site published under that GitHub account**, present and future, because GitHub Pages project sites share one hostname and differ only by path. At one user this is a footnote; it is worth writing down because it is invisible in the console and would surprise anyone who later publishes an unrelated page from the same account.

When to choose the email-link fallback instead. Only on a specific signal: her Google account turns out to be employer-managed and an administrator blocks third-party authorisation (RP-05 §6 records the same trigger from Google's OAuth documentation). Email link keeps recovery off-device and removes the Google-account prerequisite, at the cost of an inbox trip per sign-in. Note one operational detail from Firebase: the shutdown of Firebase Dynamic Links on 25 August 2025 affected email-link authentication for **mobile apps** [S10]; this app is a web page, so the web flow is the relevant one, but a maintainer reading older tutorials will hit stale instructions.

Not recommended at any point: multi-factor authentication, password rules, session timeouts. One teacher, one document of lesson dates and prices, and a sign-in that already rests on her Google account's own protections. Adding a second factor adds a second thing to lose.

2. Public-client security requirements per storage candidate

What a publishable key does and does not do

A publishable client key identifies a project to a vendor's front door. It is not an authorization credential, and treating it as one is the single most common way a static-site backend gets breached. Firebase says so directly, and the exact conditionality matters:

- **The claim, verified.** "If your app's setup follows the above guidelines, then *API keys restricted to Firebase services* do *not* need to be treated as secrets, and it's safe to include them in your code or configuration files" [S1].
- **What does the security work.** "Security of your Realtime Database, Cloud Firestore, and Cloud Storage *data* is enforced using Firebase Security Rules, and protection of covered APIs is by Firebase App Check — not by keeping your Firebase API key secret" [S1].
- **The exceptions the headline hides.** A Gemini Developer API key "should *never* be included in your code or configuration files" and must be kept out of any publicly accessible key's allowlist; keys used for non-Firebase Google Cloud APIs should be separate and restricted [S1]. The page also insists that "For **all** API keys in your project, it's critical that you make sure they have appropriate API restrictions and limits" [S1].

So the accurate statement is: *the key is publishable, provided it is restricted to Firebase services and provided the authorization rules behind it are correct*. RP-05 §6 point 2 quoted the headline; this is the full form of it.

Authorization model required per RP-05 candidate

Candidate	Authorization mechanism	Default posture	If misconfigured
Cloud Firestore	Firestore Security Rules, evaluated on every client-library request [S5]	Depends on the mode chosen at creation. <code>allow read, write: if true</code> is the pattern Firebase says "NEVER use... in production" [S3]	World read/write of every document; project id and key are in view-source
Supabase Postgres	Row Level Security plus policies, plus revoking table grants [S16]	"A table in an exposed schema without RLS is readable and writable by any role with a grant on it" [S16]	Full table read/write with the publishable key. Grants persist even after policies are added [S16]
Cloudflare Worker + KV	None exists. A Worker has no user identity; a static client must present a shared secret	n/a	The secret is in page source, so read and overwrite are open to anyone (RP-05 §2, §3B)
Google Drive <code>appDataFolder</code>	The user's own OAuth grant, scoped to the app's folder	Per-user by construction — the resource <i>is</i> her account	Blast radius is bounded by scope. The equivalent error is requesting an over-broad scope
Dropbox App folder	The user's own OAuth grant, scoped to the App folder	Per-user by construction	Same shape as Drive, plus a client-held refresh token to leak
GitHub Gist / Contents API	A personal access token = the credential	Publishing the token publishes write access	Full write, including to the deployed app in the Contents-API case

Candidate	Authorization mechanism	Default posture	If misconfigured
localStorage only	None. There is no remote surface	n/a	Nothing remote to misconfigure. The surface is the device and XSS

Two candidates deserve credit on this axis that a feature comparison usually misses. **Drive and Dropbox need no authorization rules at all**, because the data lives in the user's own account and the OAuth scope is the boundary. That is a genuine security advantage over Firestore, and it is the strongest argument in their favour that RP-05 did not make in these terms. It does not overturn RP-05's verdict, because RP-05 disqualified them on operability (a click per session for Drive, a second account for Dropbox), not on security. But if the maintainer's priority ever becomes "minimise what I am responsible for", this row is the reason to revisit.

The minimal Firestore rule for the "one document per uid" shape

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{uid} {
      allow read, write: if request.auth != null && request.auth.uid == uid;
    }
  }
}
```

Clause	What it does	What it prevents
rules_version = '2'	Pins the rules language version explicitly	Ambiguity about which semantics apply; convention, not a cited requirement
service cloud.firestore	Scopes the file to Firestore	Rules intended for Firestore silently not covering Storage, and vice versa
match /databases/{database}/documents	The mandatory root path all matches hang from	Nothing on its own; omitting it is a syntax error
match /users/{uid}	Matches one document and binds its id to the variable uid, per the documented per-user pattern [S2]	Access to any path other than a document directly under /users. Nothing matches /users itself, so no client can list the collection and enumerate user ids
allow read, write:	Grants exactly two operations	Every other operation, by omission — the default is deny
request.auth != null	Rejects callers who are not signed in; request.auth "is null" when the user is not signed in [S2]	Anyone on the internet reading or writing with just the published key

Clause	What it does	What it prevents
<code>request.auth.uid == uid</code>	The load-bearing clause. Binds the caller's identity to the document id	Any <i>other</i> signed-in Google account reading or overwriting her document

Two things this rule deliberately does not do, both worth stating so a later change does not break them silently.

It does not cover subcollections. Firebase is explicit: "Security rules apply only at the matched path, so the access controls defined on the `cities` collection do not apply to the `landmarks` subcollection", and the guidance is to "write explicit rules to control access to subcollections" [S4]. RP-05's shape has none, so today this is harmless — and because unmatched paths are denied, the failure mode of adding one later is "the feature appears broken", not "the data is exposed". That is the safe direction, but it will cost an hour of confusion if nobody remembers.

It authorizes but does not validate. A signed-in client can write any shape into its own document. That includes an injected script running in the page: RP-01 D1 runtime-verified script execution from a CSV-supplied group name, which means the same document the rules protect is writable by an attacker's payload with the user's own credentials. RP-05 §7 handles this on the read path by validating `schemaVersion` client-side. A `request.resource.data` check in the rules is the server-side equivalent and is optional at this scale — but fixing D1 is not optional, and §6 treats it as a security-of-processing measure rather than a code-quality nit.

The concrete attack surface if the rules are wrong

Discovery costs an attacker nothing. The app is at a fixed, guessable URL; the Firebase config including the project id sits in view-source; and GitHub Pages is public regardless of repository visibility [S18].

Misconfiguration	Who gets in	What they can do	How you would notice
<code>allow read, write: if true</code>	Anyone, no sign-in	Read every document, overwrite it, delete it. Firebase: "anyone who guesses your project ID can steal, modify, or delete the data" [S3]	Only from the data changing, or a quota anomaly. There is no alert by default
<code>allow read, write: if request.auth != null</code>	Any Google account in the world	Read and overwrite every user's document. Firebase's own caution: confirm "that you want any logged-in user to have access to the data" [S3]	Same — silently
Rule scoped to the wrong path	Depends	Typically over-broad reads under a wildcard	Same
Correct rule, unfixed stored XSS (RP-01 D1)	Anyone who can get a payload into a group name	Full read/write of her document, with her credentials, from her browser	Not at all

Two consequences specific to this project.

On the Spark plan the realistic harm of an open rule is denial of service, not a bill. RP-05 verified that Spark requires no payment method and that free quotas are daily ceilings. An attacker who burns the daily write allowance takes the app offline for the rest of the day; there is no card to charge. That inverts the usual threat model and it is why App Check is optional here rather than urgent — but it also means an attack is *cheap for the attacker and free of financial signal for the owner*, so the "Last synced" indicator RP-05 made mandatory is also the abuse detector.

The data that would leak is not neutral. Today the synced document would carry the payment template, which by default contains a personal IBAN and tax identifier (`index.html:387–392`). That is why RP-05's ordering constraint is a hard blocker and not a nicety: an open rule plus that template is a financial-identifier leak with a working authorization bypass in front of it.

App Check: what it adds, and why not yet

App Check "helps protect your app backends from abuse by preventing unauthorized clients from accessing your backend resources", using reCAPTCHA Enterprise on the web, and is explicitly **complementary** to both Security Rules and Firebase Authentication rather than a substitute: "Firebase Authentication provides user authentication, which protects your users, whereas App Check provides attestation of app or device authenticity" [S6]. reCAPTCHA Enterprise "is no-cost for 10,000 assessments each month, and has a cost beyond that" [S6].

Assessment for this app: **not now**. With a correct `uid == uid` rule, an attacker who lifts the config still cannot read or write her document without her Google account. What App Check would add is protection against someone hammering the project's quota with valid-looking-but-unauthorised clients — a real risk, but a hypothetical one at one user, and one whose consequence on Spark is a day of downtime rather than a bill. The switching trigger belongs next to RP-05's existing list: **enable App Check if daily usage ever deviates from what one teacher can generate**.

3. Secret-handling rules

Item	Safe to publish	Reason
Firebase <code>apiKey</code> , <code>authDomain</code> , <code>projectId</code> , <code>appId</code> , <code>messagingSenderId</code>	Y, conditionally	Firebase: keys restricted to Firebase services need not be secrets and are safe in code [S1]. Conditional on the restriction and on correct Security Rules
Firestore security-rules source	Y	Enforced server-side on every client request [S5]. Publishing them changes nothing an attacker could not probe
Google OAuth client ID	Y	A public identifier for a browser client; the redirect-URI allowlist is the control [S9]
Google OAuth client secret	N	Never needed by a browser client, and cannot be protected in one [S17][S18]
Gemini / any generative-AI API key	N	Firebase: it "should <i>never</i> be included in your code or configuration files" [S1]

Item	Safe to publish	Reason
Non-Firebase Google Cloud API key	N as a shared key	Firebase recommends separate, restricted keys for non-Firebase Google Cloud APIs [S1]
Firebase Admin SDK service-account JSON	N	Server credentials. "The server client libraries bypass all Cloud Firestore Security Rules" [S5]
Supabase publishable / anon key	Y only with RLS on	Without RLS, "A table in an exposed schema without RLS is readable and writable by any role with a grant on it" [S16]
Supabase <code>service_role</code> key	N	Bypasses RLS by design
GitHub personal access token, gist token	N	Secret scanning runs "automatically for free" on public repos and notifies the provider so it can revoke [S19]
A shared app passcode	N	Pages sites "are publicly available on the internet, even if the repository for the site is private" [S18]
Any value prefixed <code>VITE_</code>	N as a secret	Vite: such variables "should <i>not</i> contain sensitive information such as API keys" [S17]
<code>.env</code> , <code>.env.local</code> , <code>.env.*.local</code>	N	Vite: " <code>.env.*.local</code> files are local-only and can contain sensitive variables... add <code>*.local</code> to your <code>.gitignore</code> " [S17]
A real person's name, IBAN or tax identifier	N	Currently published at <code>index.html:387-392</code> and <code>400</code> . This is the report's central finding

How build-time environment variables actually behave

They are not hidden. Vite prefixes are an *exposure* mechanism, not a protection mechanism. Variables prefixed `VITE_` "will be exposed in client-side source code after Vite bundling", the values are "statically replaced at build time" and therefore become permanent parts of the production bundle, and the documentation's own security note is unambiguous: `VITE_*` variables "should *not* contain sensitive information such as API keys. The values of these variables are bundled into your source code at build time. For production deployments, consider a backend server or serverless/edge functions to properly secure secrets" [S17]. Non-prefixed variables are not exposed to client code, but they are also not *available* to it — they only exist for build tooling and any server-side code, of which this project has none.

An `.env` file therefore does exactly two useful things and one useless thing. Useful: it keeps a value out of the committed source tree (if gitignored), and it lets the value differ between environments. Useless: it does not keep the value out of the shipped artifact. Anyone who opens devtools reads it.

The practical rule for this repository

There is no build step, no bundler and no root `package.json` on `main` (verified-facts addendum; RP-01 §5), and RP-05 §7 explicitly keeps it that way by loading the Firebase modular SDK as ES modules from a CDN. So the Vite discussion above is **conditional on a future migration**, and the answer is the same

either way: embedded, not hidden.

For this repo, as it stands:

1. **The Firebase config goes in `index.html` as a literal.** No `.env`, no placeholder-substitution step, no secret-injection ceremony. Introducing an env-var mechanism to hold a non-secret would add a toolchain to protect nothing.
2. **If a value must be secret, this architecture cannot hold it.** A static page has no private execution context. Any future feature needing a real secret — a mail relay, a payment integration, a server-side API key — requires a server, which the project does not have and RP-05 declined to add. Treat that as a hard boundary when scoping features, not as a problem to engineer around.
3. **Add env-file patterns to `.gitignore` before any tooling arrives, not after.** The current `.gitignore` has six lines and covers only Node and Playwright artifacts. Adding `.env` patterns now costs nothing and removes the most common accident from the future. This is quick win QW-4.
4. **Removing a value from `HEAD` does not remove it from history.** RP-01 §8 D0 records this for the template values. It applies to any credential that is ever committed, which is why prevention beats remediation here.

4. Data sensitivity classification

Classified from RP-01 §3, which is the verified data model. Classes used: **none** (no natural person identified or identifiable), **conditional** (depends on user input), **identifiable + financial** (a named natural person plus financial and tax identifiers).

Field	Where it lives	Classification	Obligation trigger
<code>Group.name</code>	<code>groupLessonPlannerData</code>	Conditional. Free text. None if a schedule label; personal data if a person's name	Only when the value identifies a person. This is the single field that can switch GDPR on
<code>Group.price</code>	same	None. The teacher's own commercial pricing	None
<code>Group.currency</code>	same	None	None
<code>Group.dates</code>	same	None alone. Sensitivity is inherited from the group name	Only via an identifying name; then it reveals when a named person attends
<code>Group.monthlyOverrides[m].price</code>	same	None	None
<code>Group.monthlyOverrides[m].dates</code>	same	None alone; same inheritance as <code>dates</code>	As above
<code>Settings.defaultCurrency</code>	<code>groupLessonPlannerSettings</code>	None	None

Field	Where it lives	Classification	Obligation trigger
Payment template	paymentTemplate, default at index.html:382-400	Identifiable + financial. A natural person's full name, personal bank IBAN and tax identification number at 387-392 ; a personal first name at 400	Yes, unconditionally, today. Published in view-source and in git history

Three points this table is designed to make

The dates are only as sensitive as the name. A list of ISO dates with a price is a scheduling artefact. The same list under the heading of a child's full name is a record of a minor's weekly whereabouts. This is why §6's highest-leverage measure is a naming convention rather than a cryptographic control: change the label and the whole subtree drops to "none".

The absence of fields is the app's biggest compliance asset, and it was free. There is no email, no phone, no address, no parent contact, no notes field, no attendance flag, no payment-received flag, and no special-category data of any kind (nothing in Article 9(1)'s list — racial or ethnic origin, political opinions, religious or philosophical beliefs, trade union membership, genetic data, biometric data for unique identification, health, sex life or sexual orientation [S21]). That is data minimisation by design in the sense of Article 25 [S20], achieved by accident of scope. **The cheapest data-protection decision available is to keep it that way**, and the most expensive future decision would be to add a "parent phone number" field.

The template is a different kind of problem from the rest of the table, and conflating them produces bad advice. The group-name question is "might GDPR obligations attach to processing about students?". The template question is "a real person's bank and tax identifiers are on the public internet". The second is not primarily a compliance failure toward a third party — it is a fraud and identity-theft exposure, and it is already realised. Note the wrinkle that makes it a compliance question too: the maintainer and the end user are different people, so if the named person is the teacher, the maintainer published a third party's financial identifiers to an indefinite audience. That is the Lindqvist fact pattern (§5), and it is the one place where the household exemption is most clearly unavailable.

How "none" becomes a leak anyway

Three verified paths, all from RP-01, all worth listing because they are how a low-sensitivity dataset ends up somewhere it should not be: stored XSS from a CSV-supplied group name executing script in the page (D1); a CSV export that omits the template but carries every group name in plaintext to wherever the file goes (D16, D17); and the clipboard, which receives the fully substituted payment message including the payment details on every copy.

5. GDPR analysis

Poland is the user's location, so the Polish supervisory authority (UODO) and the GDPR as directly applicable EU law are the relevant frame. Read §10 before relying on anything here.

5.0 Who is the controller, and why it matters before anything else

The teacher decides what to type into the app. The maintainer decides what the app is, where the bytes go and who the processor is. Under Article 4(7) the controller is the one who determines the purposes and means of processing [S20], and on these facts those two determinations are split between two people.

This matters because of one sentence in Recital 18. The recital describes personal or household activity as covering things like correspondence, address books, social networking and online activity in personal contexts — and then states that the Regulation **does** apply to controllers or processors that *provide the means* for such personal-data processing [S20]. So even on the most generous reading of the exemption for the teacher, **the exemption does not travel to the person who built and hosts the tool and owns the Firebase project**. Anyone reaching for "it's just a personal project" as a shield should read that sentence first.

The honest position is that the controller/processor allocation on these facts is unsettled — see the contested list below. The EDPB's guidance on the concepts is Guidelines 07/2020, final version adopted 7 July 2021 [S26]; this report verified its title and adoption date but could not extract its text (the published PDF did not parse), so no substantive claim is attributed to it.

5a. The household exemption: does it apply, and where does it stop

Step 0 — the question that comes first. Is there personal data at all? Article 4(1) makes information personal data when it relates to an identified or identifiable natural person, with name, identification number, location data and online identifiers among the listed identifiers [S20]. A group called *Tuesday intermediate* relates to no identifiable person. On the verified data model (§4), **if group names are non-identifying, GDPR does not engage for the student-facing data**, and no exemption is needed. That is a cleaner and stronger position than any argument about household activity, and it is achievable by a naming convention (§6).

Step 1 — the exemption itself, if she does type a name. Article 2(2)(c) excludes "processing of personal data by a natural person in the course of a purely personal or household activity" [S20]. Two CJEU judgments define how narrow that is:

- **Lindqvist (C-101/01, 6 November 2003).** The exception must be interpreted "as relating only to activities which are carried out in the course of private or family life of individuals", which is "clearly not" the case for publication on the internet making data accessible to an indefinite number of people (para. 47). The Court reached that result even though the underlying activity was unpaid parish volunteering — so **the absence of an economic motive does not save the exemption** [S22].
- **Ryneš (C-212/13, 11 December 2014).** The exception "must be narrowly construed"; the word "purely" signals a restrictive scope; it applies "only where it is carried out in the purely personal or household setting"; and surveillance "directed outwards from the private setting" covering public space "cannot be regarded as an activity which is a purely 'personal or household' activity" (paras. 27-33) [S23].

UODO applies the same narrow reading. Its 8 December 2025 decision on a neighbour's cameras frames the exclusion as processing "w ramach sfery o czysto osobistym lub domowym charakterze" ("within a sphere of a purely personal or domestic character") and holds it unavailable "o ile nadzór kamer wideo

rozciąga się choćby częściowo na przestrzeń publiczną" ("insofar as video camera surveillance extends even partially to public space") [S27].

Application here. Paid group teaching is a professional, remunerated activity, not domestic life. Records of who attends which paid lesson and what they owe are ordinary commercial client records; the whole point of the app is to produce an invoice-like message. If a group name identifies a student, the exemption most likely fails. **Label: this is a reasoned application of the case law to new facts, not a settled holding.** No CJEU or UODO decision on private tutoring records was found; both leading cases concern publication and surveillance, not a private client list.

Where it stops — four boundaries, all sourced. (i) Publication to an indefinite audience (Lindqvist para. 47 [S22]). (ii) Processing reaching outside the private sphere (Ryneš paras. 31-33 [S23]; UODO [S27]). (iii) Provision of the means, which is never covered (Recital 18 [S20]). (iv) Connection to an economic or professional activity — the weakest-sourced of the four, because Lindqvist shows the exemption can fail *without* an economic activity, so a commercial link is sufficient to defeat it but its absence is not sufficient to save it.

And it plainly does not cover the template exposure. Publishing a named person's IBAN and tax identifier on a public website reaching an indefinite audience is squarely inside Lindqvist's holding. Whatever is true of the teacher's group list, boundary (i) disposes of this one.

5b. What attaches if the exemption does not apply

Obligation	Source	What it means at this scale
Lawful basis	Art. 6(1): consent, contract, legal obligation, vital interests, public task, legitimate interests [S20]	Contract (6(1)(b)) for the teaching arrangement, or legitimate interests (6(1)(f)) for admin records. Not consent — it is revocable, and she needs the records to invoice
Purpose limitation, minimisation, accuracy	Art. 5(1)(b)-(d) [S20]	Already satisfied by the data model. Do not add fields
Storage limitation	Art. 5(1)(e) [S20]	The one principle the app currently fails: nothing is ever deleted. Needs a stated retention period and a way to act on it
Integrity and confidentiality	Art. 5(1)(f), which expressly covers "accidental loss, destruction or damage" [S21]; Art. 32(1), which names pseudonymisation and encryption as examples [S20]	The failure already suffered was availability, not confidentiality. RP-05's fallback — error handling plus a versioned backup — is a security-of-processing measure, not just a reliability nicety
Transparency	Art. 13(1) and (2): controller identity, purposes, legal basis, recipients, retention, rights, right to complain [S20]	One sentence to parents. Not a document

Obligation	Source	What it means at this scale
Data subject rights	Arts. 15-22: access, rectification, erasure, restriction, portability, objection, automated decisions [S20]	Satisfied operationally: she can open a group, edit it, delete it, and export a JSON file. Article 22 is irrelevant — nothing is automated
Accountability	Art. 5(2): the controller must be able to demonstrate compliance [S21]	One paragraph of internal record. See §7
By design and by default	Art. 25 [S20]	Already the app's strongest property; §6 measure 3 is the cheapest way to strengthen it
Breach notification	Art. 33(1): without undue delay and where feasible within 72 hours; Art. 34(1) to data subjects on high risk [S20]	The obligation exists if a notifiable breach happens. A standing runbook does not need to

On records of processing, get the exemption right. Article 30(5) removes the record-keeping duty for an enterprise or organisation employing fewer than 250 persons — **unless** any of four things is true: the processing is likely to result in a risk to the rights and freedoms of data subjects; **the processing is not occasional**; it includes Article 9(1) special categories; or it includes Article 10 criminal-conviction data [S21]. Ongoing term-by-term client records are, on any natural reading, *not occasional*. So the derogation quite plausibly does **not** apply, and a record is technically required. The proportionate response is not a compliance programme and not a shrug: **it is one short paragraph** — what is processed, why, on what basis, where it is stored, who the processor is, how long it is kept. That paragraph also doubles as the source text for the Article 13 sentence. §7 makes it item 7.

On registration: there is nothing to register. Recital 89 records that the general notification obligation under Directive 95/46 produced administrative and financial burdens, did not in all cases improve protection, and that the Regulation abolishes those indiscriminate general notification obligations, replacing them with mechanisms focused on high-risk processing [S20]. There is no filing to make with UODO for this app.

5c. Controller/processor with a hosted vendor, and the Article 28 contract

The requirement. Article 28(1) requires that processing by a processor be governed by a contract or other legal act binding the processor to the controller, and Article 28(3) lists what it must contain — subject matter and duration, nature and purpose, type of personal data, categories of data subjects, the parties' obligations, and the processor's duty to return or delete the data at the end [S20].

What Google offers, verified. The Firebase Data Processing and Security Terms (last modified 21 August 2024) state at §5.1.1 that "Google is a processor of that Customer Personal Data under European Data Protection Law; Customer is a controller or processor, as applicable", reference Article 28(3) GDPR directly, incorporate the EU Standard Contractual Clauses in controller-to-processor, processor-to-controller and processor-to-processor variants, and point to a subprocessor list [S12]. The Cloud Data Processing Addendum (Customers) states the equivalent at §4.1 [S13].

How it is accepted. Not by signature and not by a separate click. Both documents say they are "incorporated into the Agreement(s)" between Google and the customer, and take effect from the date the customer accepted, or the parties otherwise agreed to, the terms [S12][S13]. In practice: accepting the Firebase / Google Cloud terms when the project is created is the acceptance. **There is nothing to negotiate on a free plan, and nothing to sign** — which is why §7 lists "negotiate a bespoke DPA with Google" under *not required*.

The open question, stated rather than guessed. Neither document states whether it applies to no-cost use. The Firebase terms define "Services" broadly and reference "Paid Services, APIs or Services (as applicable)" without a payment-status carve-out, and the Cloud addendum does not address free tiers at all [S12][S13]. **This report does not assert that the DPA covers Spark-plan usage.** It is recorded as an open question in §8 with what would resolve it. It does not block anything: if the answer turned out to be no, the consequence would be an argument for the RP-05 fallback, which involves no processor at all.

Subprocessors, which is the part people skip. The Firebase subprocessors page (last modified 23 September 2021) lists Accenture LLP, EPAM Systems Inc, Fastly Inc and Firebase Inc — **all in the United States** — for support, content delivery and infrastructure, and incorporates the Google Cloud Platform subprocessor list by reference [S14]. So even with the database pinned to Warsaw, the processing chain includes US entities. That is not a defect; it is the fact pattern that 5d has to deal with.

What the maintainer actually has to do. Nothing contractual. Three practical things: write down who the controller is (the paragraph from 5b); keep the Admin SDK off any machine he does not control, since server client libraries "bypass all Cloud Firestore Security Rules" [S5]; and re-read the subprocessor list if the app's data ever becomes more sensitive than lesson dates.

5d. Residency and international transfers per candidate

Candidate	Data at rest	Transfer position	Assessment
Firestore europe-central2	Warsaw, Poland; "once you provision a database instance, you cannot change its location setting" [S11]	Google LLC is DPF-certified and also relies on SCCs [S15]. Art. 45 adequacy via Decision (EU) 2023/1795 [S24]	Best of the hosted options. Choose the region at creation; there is no second chance
Supabase eu-central-1	Frankfurt (RP-05 §3C)	Not researched here; excluded on durability grounds in RP-05 §4	Moot
Cloudflare Worker + KV	Globally replicated, no residency control documented (RP-05 §3C)	Would need its own analysis	Rejected earlier on authorization grounds anyway
Google Drive appDataFolder	Consumer Google account; region not user-selectable — TBD (RP-05 §8 left the same item open)	Same DPF/SCC position for Google LLC [S15]	Weaker on residency than Firestore, and no better on transfers
Dropbox App folder	TBD — not researched	TBD	Moot; excluded on operability

Candidate	Data at rest	Transfer position	Assessment
localStorage + JSON file	Her own disk, in Poland	No transfer at all. No processor at all	The strongest possible answer on this axis, and it is RP-05's fallback

The transfer analysis for Firestore, in full. Commission Implementing Decision (EU) 2023/1795 of 10 July 2023 finds that the United States ensures an adequate level of protection for personal data transferred to organisations certified under the EU-U.S. Data Privacy Framework [S24]. Google states that "Google LLC (and its wholly-owned US subsidiaries unless explicitly excluded) has certified that it adheres to the DPF Principles", covering the EU-U.S. DPF, the Swiss-U.S. DPF and the UK Extension, and adds that it relies on SCCs "for our data transfers where required and in instances where they are not covered by an adequacy decision" (page effective 23 August 2025) [S15]. Under Article 45, a transfer to an adequate destination needs no further authorisation; Article 46 safeguards including standard contractual clauses are the fallback where no adequacy decision applies [S20]. So the position is: **primary route adequacy, backstop SCCs, both already in place without action by the maintainer.**

And the contested part, because it is contested. The adequacy decision was challenged and survived at first instance: in Case T-553/23 *Latombe v Commission*, judgment of 3 September 2025, the General Court dismissed the action for annulment of Decision (EU) 2023/1795, rejecting all pleas and holding that the Data Protection Review Court meets the required standard [S25]. That is first-instance, and an appeal to the Court of Justice has been reported in secondary sources; this report could not confirm the appeal from a primary source and records it as an open item in §8. The practical exposure if adequacy were ever struck down is smaller here than the headline suggests, for two reasons: the data at rest stays in Warsaw [S11], and Google states it relies on SCCs where an adequacy decision does not cover a transfer [S15]. The residual exposure is support and CDN subprocessing by US entities [S14].

Verified legal requirements

Each item below is what a primary text or judgment says, not an application of it.

Requirement	Source
Art. 2(2)(c) excludes processing by a natural person "in the course of a purely personal or household activity"	[S20]
Recital 18: the Regulation applies to controllers/processors providing the means for such personal or household processing	[S20]
The exemption "must be narrowly construed"; "purely" signals restrictive scope; it applies only in the purely personal or household setting	Ryneš paras. 27-33 [S23]
The exemption relates "only to activities... carried out in the course of private or family life"; internet publication to an indefinite audience is "clearly not" that	Lindqvist para. 47 [S22]
Art. 4(1) definition of personal data; Art. 4(7) controller; Art. 4(8) processor; Art. 3(1) territorial scope	[S20]
Art. 5(1) six principles; Art. 5(1)(f) expressly covers "accidental loss, destruction or damage"; Art. 5(2) accountability	[S20][S21]

Requirement	Source
Art. 6(1) six lawful bases	[S20]
Arts. 13(1) and 13(2): the information to be given when data is collected from the data subject	[S20]
Arts. 15-22 data subject rights	[S20]
Art. 25 data protection by design and by default	[S20]
Art. 28(1) requires a contract with a processor; Art. 28(3) lists its required content	[S20]
Art. 30(5): derogation below 250 persons, cancelled by risk to rights, processing that is not occasional , Art. 9(1) data, or Art. 10 data	[S21]
Art. 32(1) security of processing, naming pseudonymisation and encryption as examples	[S20]
Arts. 33(1) and 34(1) breach notification, including the 72-hour rule	[S20]
Art. 35(3) DPIA cases; Art. 37(1) DPO cases	[S20]
Art. 9(1) list of special categories	[S21]
Arts. 44-46 transfer regime, adequacy and appropriate safeguards including SCCs	[S20]
Recital 89: the Regulation abolishes indiscriminate general notification obligations	[S20]
Adequacy for the US exists for DPF-certified organisations, by Decision (EU) 2023/1795 of 10 July 2023	[S24]
The General Court dismissed the annulment action against that decision on 3 September 2025	[S25]
Google is a processor and the customer is controller (or processor); the terms reference Art. 28(3) and incorporate SCCs; they are incorporated into the underlying agreement rather than separately signed	[S12][S13]
UODO applies the narrow reading of the exclusion and treats it as unavailable once processing extends beyond the private sphere	[S27]

Contested points

Point	The disagreement	Which way the weight falls
Whether a private teacher's paid client records fall inside the household exemption	No CJEU or UODO decision on point. Lindqvist and Ryneš both concern reaching <i>outside</i> the private sphere — publication and public-space surveillance — not a private client list kept on one device	Against the exemption, because the activity is remunerated and professional and the case law construes the exclusion narrowly. But the fact pattern is genuinely undecided, and a private notebook of clients shown to nobody is distinguishable from both leading cases

Point	The disagreement	Which way the weight falls
Whether the teacher or the maintainer is the controller once the maintainer owns the Firebase project	Art. 4(7) turns on who determines purposes and means, and here the two are split between two people. Joint control (Art. 26) is arguable	Unresolved. Recital 18 at least settles that the maintainer cannot claim the household exemption for providing the means
Durability of EU-US adequacy	Upheld at first instance in T-553/23 [S25]; an appeal is reported in secondary sources only	Adequate today. The practical hedge already exists: EU-region storage plus Google's stated SCC reliance [S11][S15]
Whether "API keys are not secrets" is a safe general rule	Firebase says yes for Firebase-restricted keys, and names exceptions in the same page [S1]. Vite says the opposite about secrets generally [S17]	No real conflict. The Firebase config is an identifier; a secret is a credential. Confusing the two is what breaks

Open questions

Collected with everything else in §8.

6. Minimisation measures

Measure	Risk reduced	Usability cost	Recommended
1. Remove the personal payment details from the default template (<code>index.html:387–392</code> , <code>400</code>)	The only certain, already-realised exposure. Also unblocks RP-05 §7	Near zero. She types her own details once, into a field she can already edit	Y — first
2. Treat the disclosed IBAN and tax id as public, and tell the named person	Fraud and identity theft. Nothing in code can undo publication	None. It is a conversation	Y — highest real-world value
3. Non-identifying group labels as the working convention	Turns <code>Group.name</code> and, by inheritance, all date fields from "conditional" to "none" (§4)	Zero to negative — <code>Tue 18:00 B1</code> is easier to scan than a name	Y — highest leverage per unit of effort
4. Never add a contact field	Prevents the category of data the app has so far avoided entirely	Zero, as a design constraint	Y

Measure	Risk reduced	Usability cost	Recommended
5. Fix the stored XSS (RP-01 D1)	Art. 32 security. Strictly more important with sync: injected script inherits her Firestore session	Zero. Escaping is invisible for benign names	Y
6. Storage error handling plus a versioned JSON backup (RP-05 fallback)	Art. 5(1)(f) "accidental loss, destruction or damage" [S21] — the failure that already happened	Positive: it adds a backup she does not have	Y
7. Stated retention period plus an explicit "delete months before X" action	Art. 5(1)(e) storage limitation. Nothing is ever deleted today	Low, but it destroys data, so it needs a confirm	Y, low priority
8. Make <code>Clear All Data</code> also clear <code>paymentTemplate</code> (RP-01 D15)	The confirm text already promises a full wipe, and the key it leaves behind is the one holding personal data	None — it is what the dialog claims	Y , but it is a product decision (RP-01 open question 4), so not a quick win
9. One-sentence privacy note to parents (Art. 13)	Transparency, if any name is ever stored	Trivial	Conditional Y — only once a name is stored
10. Purge the values from git history	Reduces casual future discovery	A force-push; forks, caches and mirrors may retain copies	Y, with a warning — mitigation, never remediation
11. Firebase App Check	Quota abuse from unauthorised clients [S6]	None to her; free to 10,000 assessments/month [S6]	N for now. Trigger in §2
12. Client-side encryption of the free-text group name	Almost nothing, in this architecture	High, and it destroys recoverability	N — this is theatre. See below
13. Ship no sync at all (RP-05 fallback only)	Removes the processor, the transfer question and the DPA question entirely	No multi-device; depends on her doing backups	Conditional — it ships first regardless

Why client-side encryption is theatre here, stated plainly

The brief asks for honesty about which measures are protection and which are performance. Measure 12 is performance.

Encrypting the group name requires a key, and in a static single-page app there are only three places to put it. **In the browser:** the key then dies with the profile, which is precisely the event that destroyed the data in the first place — encryption would convert a recoverable dataset into an unrecoverable one.

Derived from a passphrase she types: a non-technical user with no recovery path, and the moment she

forgets it the cloud copy is landfill; this trades a small confidentiality gain for a large availability loss, and availability is the requirement that started this programme. **Stored next to the ciphertext in Firestore:** self-evidently pointless.

And even a correctly-keyed scheme protects against a narrow threat: the vendor, or a rules misconfiguration. It does nothing about the realistic paths, because the plaintext must exist in the DOM to be displayed, and RP-01 D1 verified that script already executes in that DOM. Encrypting a field while shipping an XSS is a lock on a door with no wall.

The honest substitute is measure 3. Not storing an identifying string at all is strictly better than encrypting one: it is free, it cannot lose a key, it cannot be undone by an XSS, and it moves the field from "conditional" to "none" in §4. Pseudonymisation is named in Article 32(1) as an example of an appropriate measure [S20]; a naming convention is the cheapest pseudonymisation there is.

Two more honest notes. **Measure 10 is not remediation.** Values that have been publicly readable since the first commit should be assumed collected; rewriting history reduces discovery, not disclosure, which is why measure 2 outranks it. And **measure 7 must not be automated.** An app with no storage error handling and a verified history of total data loss should not acquire a background deletion job; make it an explicit, confirmed user action or leave it out.

7. Recommended proportionate posture, with an explicit "not required here" list

Do, in this order

#	Action	Why now	Effort
1	Remove the personal payment details from the default template	The only realised exposure; blocks RP-05 §7	XS
2	Tell the named person the identifiers were public, and let them decide about the bank account	Nothing in code substitutes for this	None (a conversation)
3	Escape <code>group.name</code> in the card sink (<code>index.html:1047</code>)	Art. 32 security; becomes a credential-theft path once sync exists	XS
4	Storage error handling plus a versioned JSON backup (RP-05 fallback)	Art. 5(1)(f) covers accidental loss [S21]. This is the failure that already happened	S
5	Adopt non-identifying group labels; add no contact fields	Keeps §4 at "none" and keeps §5 out of scope	XS
6	When sync ships: Google sign-in via <code>signInWithPopup</code> , the four-line rule, <code>europe-central2</code> , <code>denyslystopadsky.github.io</code> authorized	§1 and §2. The popup flow is a requirement, not a preference [S8]	S

#	Action	Why now	Effort
7	Write one paragraph: what is processed, why, on what basis, where, how long, who the processor is	Art. 5(2) accountability, plus Art. 30 if the derogation fails [S21]. Doubles as the Art. 13 text	XS

Optional, with the trigger that would select it

Measure	Trigger
App Check	Daily Firestore usage deviates from what one teacher can generate
Email-link sign-in as an alternative provider	Her Google account turns out to be employer-managed and authorisation is blocked
Retention period plus an explicit delete-old-months action	The dataset grows past a few teaching years, or she asks
A one-sentence privacy note to parents	Any stored group name identifies a person
Git-history purge	Decided alongside item 2, and only after it
A rules-level <code>request.resource.data</code> shape check	A second writer, or a second app, ever touches the document

Not required here — with the reason, not the assertion

Not required	Reason
Data Protection Officer	Art. 37(1) lists three cases: public authority; core activities requiring regular and systematic large-scale monitoring; core activities involving large-scale special categories or criminal-conviction data [S20]. None applies
Data Protection Impact Assessment	Art. 35(3) lists systematic extensive automated evaluation with legal effect, large-scale Art. 9/10 processing, and large-scale systematic monitoring of a public area [S20]. None applies. RP-01 measured the whole dataset at 354 bytes
Registration or notification to UODO	Recital 89: the Regulation abolishes indiscriminate general notification obligations [S20]. There is nothing to file
Cookie or consent banner	RP-01 runtime-verified zero cookies, zero third-party requests and zero analytics. Adding Firebase Auth introduces storage for a sign-in the user explicitly asked for; the ePrivacy analysis of that is out of scope for this report and is listed in §8
A negotiated, signed DPA with Google	The terms are incorporated into the underlying agreement rather than separately signed, and already reference Art. 28(3) and the SCCs [S12] [S13]. There is no counterparty to negotiate with on a free plan

Not required	Reason
A standing breach-notification runbook	Arts. 33/34 obligations apply if a notifiable breach occurs [S20]. What is disproportionate is a documented 72-hour process for a one-document app. Item 7 above should name who to contact, in one line
Client-side encryption of free text	§6 measure 12. It is theatre, and it trades away recoverability
Multi-factor authentication, password policy, session timeout	§1. One user, one Google account, and MFA is already available to her at the Google-account level if she wants it
Anonymous auth, a device-bound key, or an unguessable-URL scheme	§1. Each reproduces the device-loss failure that started this programme
Formal vendor security assessment, penetration test, SIEM, audit logging, ISO 27001-style controls	Sized for an organisation. Here the entire attack surface is one static page, one Firestore document, and one Google account
A separate privacy-policy website and a consent management platform	There is no third-party tracking, no marketing, and no second user to inform
Restricting the Firebase API key to specific referrers as a <i>security</i> control	Worth doing as hygiene [S1], but it is not what protects the data — the rules are [S1]. Do not let it substitute for §2

The one obligation that plausibly does bite, and is easy to miss. If a group name ever identifies a student, Articles 13(1) and 13(2) transparency applies [S20] — the parents are entitled to be told who holds what, why, and for how long. The proportionate discharge is one sentence appended to whatever she already sends about scheduling and fees, drawn from item 7's paragraph. That is the whole obligation at this scale. It is not a programme, and it is also not nothing.

8. Risks and unknowns

Risks in this report's own recommendations

Risk	Severity	Mitigation
Sync ships before the template is cleaned, replicating an IBAN and tax id into a third-party datastore and its backups	Critical	Hard blocker, as in RP-05 §7-§8. §7 item 1
Google sign-in implemented with <code>signInWithRedirect</code> and silently failing on her browser	High	Use <code>signInWithPopup</code> ; Firebase documents the requirement for apps off the Firebase auth domain [S8]
A rule written as <code>request.auth != null</code> without the uid comparison	High	§2's clause table. Firebase flags the pattern itself [S3]

Risk	Severity	Mitigation
Adding a subcollection later under a document-level match	Medium	Rules "apply only at the matched path" [S4]. Unmatched paths deny, so the symptom is breakage, not exposure
Stored XSS (RP-01 D1) left unfixed once a signed-in session exists in the page	High	§7 item 3. RP-05 §8 records the same risk
Authorized-domain grant covering every GitHub Pages project on the account	Low	Accept and record it [S9]; revisit if the account ever publishes something untrusted
Treating a git-history purge as remediation for the disclosed identifiers	Medium	§6 measure 10. Treat the values as public
Automating retention deletion in an app with no error handling	Medium	§6 measure 7: explicit, confirmed user action only

Everything marked TBD or open

Item	Why unresolved	What resolves it
Which browser and device the teacher uses — TBD , and this report promotes it	RP-05 §8 already called this its highest-value unknown, on session-persistence grounds. [S8] escalates it: the redirect workaround is <i>already required</i> on Firefox 109+ and Safari 16.1+, so the answer now decides whether the sign-in flow works at all, not merely how often she signs in again	One question to the teacher: Safari, Chrome, Edge or Firefox, on a Mac, a Windows laptop or an iPad. Then confirm the popup flow on that browser before shipping
Whether the Firebase / Cloud DPA applies to no-cost Spark usage	Neither document addresses payment status; "Services" is defined broadly and the Cloud addendum is silent [S12][S13]	Read the Firebase Terms of Service and the Google Cloud Platform Terms of Service for whether the addendum is incorporated for no-charge services, or look for a console acceptance surface
Whether the person named at <code>index.html:387–392</code> and <code>400</code> is the end user or a third party	Not stated in any input, and this report did not attempt to identify them	Ask the maintainer. It determines whether the publication was of his own data or someone else's
Whether the maintainer is established in the EU	Not in any input. Art. 3(1) turns on establishment [S20]	Ask the maintainer
Whether any current or future group name identifies a person	The teacher's dataset was destroyed with the laptop; no artefact survives (RP-01)	Ask the teacher what she called her groups. §6 measure 3 makes the answer moot going forward

Item	Why unresolved	What resolves it
Retention period she actually needs — TBD	Driven by Polish tax and accounting record-keeping rules, which this report did not verify	Her accountant, or the Polish tax and accounting statutes. Do not guess a number
Appeal status of the DPF adequacy litigation — TBD	An appeal to the Court of Justice is reported only in secondary sources; the primary record consulted shows no appeal [S25]	Search InfoCuria for an appeal registered against T-553/23
Substance of EDPB Guidelines 07/2020 on controller and processor	Title and adoption date (7 July 2021) verified; the published PDF did not parse [S26]	Fetch a text-extractable copy, or the HTML rendering, and read the controller-determination section
ePrivacy consent position for Firebase Auth session storage	Out of scope for this report; the ePrivacy Directive was not consulted	Read Art. 5(3) of Directive 2002/58/EC as implemented in Polish law, plus any UODO guidance on strictly-necessary storage
Google Drive consumer-account data residency — TBD	Consumer accounts expose no region choice; RP-05 §8 left the same item open	Likely unanswerable in the user's favour; Workspace data-regions documentation does not apply to consumer accounts
Dropbox EU residency and token lifetime — TBD	Not researched; the candidate was excluded on operability	Provider trust-centre and the <code>expires_in</code> field from the token endpoint
Exact Firestore free-quota figures — TBD for this report	Taken from RP-05 §3A rather than re-fetched by me	Firebase pricing and Firestore quotas pages

Where an input was wrong or incomplete

Input	Finding
The research pack's premise that the app stores data about identified students	Wrong. RP-01 §3 verified there is no student entity. The only field that can carry a student's identity is the free-text group name, and nothing forces it to. This is the correction that reshapes §4 and §5
RP-05 §6, which argued Google sign-in from persistence and token models	Incomplete, not wrong. It did not analyse recovery, which is the axis the actual incident tested and the one that most decisively rules out passkeys, anonymous auth and capability URLs
RP-05 §7, which specifies Google sign-in without naming the flow	Incomplete. <code>signInWithRedirect</code> is not viable on GitHub Pages without a documented workaround [S8]. The implementation must specify <code>signInWithPopup</code> and the authorized-domain entry
RP-05 §6 point 2, quoting "API keys ... do not need to be treated as secrets"	Correct but truncated. The Firebase page conditions that on the app following its guidelines and names exceptions where a key must never be published [S1]

Input	Finding
A common reading of Art. 30(5) as a blanket "<250 employees" exemption	Wrong. Four conditions cancel it, including "the processing is not occasional" [S21], which ongoing client records plausibly fail. §5b states the proportionate consequence
The contract's citation of the personal data at <code>index.html:386–392</code>	Confirmed off by one at the start and missing line <code>400</code> , exactly as RP-01 §8 and RP-05 §8 recorded. Line 386 is a heading; the details are at 387–392; a personal first name is at 400. Independently re-checked here by a redacting line-length scan

9. Sources

#	Title	URL	Accessed	Supports
S1	Firebase — Learn about using and managing API keys	https://firebase.google.com/docs/projects/api-keys	2026-08-20	Firebase-restricted keys need not be secrets, conditionally; the Gemini and non-Firebase Cloud API exceptions; Security Rules and App Check do the security work; apply API restrictions anyway
S2	Firebase — Security Rules and Firebase Authentication	https://firebase.google.com/docs/rules/rules-and-auth	2026-08-20	<code>request.auth</code> contents and that it is null when the user is not signed in; the recommended per-user ownership pattern matching <code>request.auth.uid</code> to the document id
S3	Firebase — Fix insecure rules	https://firebase.google.com/docs/rules/insecure-rules	2026-08-20	The <code>if true</code> pattern and the warning never to use it in production; that anyone who guesses the project id can steal, modify or delete data; the caution on <code>auth != null</code>
S4	Firebase — Structure Cloud Firestore Security Rules	https://firebase.google.com/docs/firestore/security/rules-structure	2026-08-20	Rules apply only at the matched path and do not cascade to subcollections; the recursive wildcard; rules are access controls, not filters
S5	Firebase — Get started with Cloud Firestore Security Rules	https://firebase.google.com/docs/firestore/security/get-started	2026-08-20	Every client-library request is evaluated against rules; server client libraries bypass all rules and authenticate through application default credentials

#	Title	URL	Accessed	Supports
S6	Firebase App Check	https://firebase.google.com/docs/app-check	2026-08-20	What App Check protects against; reCAPTCHA Enterprise as the web provider with 10,000 no-cost assessments a month; that it is complementary to Security Rules and does not authenticate users
S7	Firebase — Authenticate Using Google with JavaScript	https://firebase.google.com/docs/auth/web/google-signin	2026-08-20	Enabling the Google provider in the console; popup and redirect flows both offered
S8	Firebase — Best practices for signInWithRedirect flows	https://firebase.google.com/docs/auth/web/redirect-best-practices	2026-08-20	Third-party storage blocking breaks <code>signInWithRedirect</code> ; required on Chrome M115+, already required on Firefox 109+ and Safari 16.1+; <code>signInWithPopup</code> avoids it; apps off the Firebase auth domain must act
S9	Firebase Help — Set a web app's OAuth redirect domains	https://support.google.com/firebase/answer/6400741	2026-08-20	Only <code>localhost</code> and the project's hosting domain are authorized by default; a domain must be whitelisted; whitelisting allows requests from any URL and port of that domain
S10	Firebase — Authenticate with Firebase Using Email Link	https://firebase.google.com/docs/auth/web/email-link-auth	2026-08-20	How email-link sign-in works; the same-device and email-match requirements; not passing the email in redirect parameters; the Dynamic Links shutdown affecting mobile email-link auth
S11	Cloud Firestore — Locations	https://firebase.google.com/docs/firestore/locations	2026-08-20	<code>europe-central2</code> is Warsaw; once a database instance is provisioned its location cannot be changed
S12	Firebase Data Processing and Security Terms	https://firebase.google.com/terms/data-processing-terms	2026-08-20	Google is a processor and the customer a controller or processor; the Art. 28(3) reference; the SCC variants; incorporation into the Agreement rather than separate signature; the subprocessor list pointer; last modified 21 August 2024
S13	Google — Cloud Data Processing Addendum (Customers)	https://cloud.google.com/terms/data-processing-addendum	2026-08-20	Automatic incorporation into the underlying agreement; §4.1 processor/controller allocation; silence on free-tier applicability

#	Title	URL	Accessed	Supports
S14	Firebase Subprocessors	https://firebase.google.com/terms/subprocessors	2026-08-20	The named subprocessors and that all listed ones are in the United States; incorporation of the Google Cloud Platform subprocessor list; last modified 23 September 2021
S15	Google — Data transfer frameworks	https://policies.google.com/privacy/frameworks	2026-08-20	Google LLC and wholly-owned US subsidiaries certified under the EU-U.S. DPF, Swiss-U.S. DPF and UK Extension; reliance on SCCs where adequacy does not cover a transfer; effective 23 August 2025
S16	Supabase — Row Level Security	https://supabase.com/docs/guides/database/postgres/row-level-security	2026-08-20	A table in an exposed schema without RLS is readable and writable by any role with a grant; adding policies does not remove grants; the <code>auth.uid()</code> ownership policy pattern
S17	Vite — Env Variables and Modes	https://vite.dev/guide/env-and-mode	2026-08-20	<code>VITE_</code> variables are exposed in client source after bundling; they must not contain sensitive information; values are statically replaced at build time; <code>.env.*.local</code> must be gitignored
S18	GitHub Docs — Securing your GitHub Pages site with HTTPS	https://docs.github.com/en/pages/getting-started-with-github-pages/securing-your-github-pages-site-with-https	2026-08-20	Pages sites are publicly available on the internet even if the repository is private; they should not be used for sensitive transactions; HTTPS enforcement redirects all HTTP requests
S19	GitHub Docs — About secret scanning	https://docs.github.com/en/code-security/secret-scanning/introduction/about-secret-scanning	2026-08-20	Secret scanning runs automatically and free on public repositories; detected partner secrets are reported to the provider so it can revoke them

#	Title	URL	Accessed	Supports
S20	Regulation (EU) 2016/679 (GDPR)	https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?uri=CELEX:32016R0679	2026-08-20	Arts. 2(2)(c), 3(1), 4(1), 4(7), 4(8), 5(1), 6(1), 13(1) and 13(2), 15-22, 25, 28(1), 28(3), 32(1), 33(1), 34(1), 35(1), 35(3), 37(1), 44-46; Recitals 18 and 89
S21	Regulation (EU) 2016/679 — consolidated text	https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX%3A02016R0679-20160504	2026-08-20	The four conditions cancelling the Art. 30(5) derogation including "the processing is not occasional"; Art. 5(1) (f) covering accidental loss, destruction or damage; Art. 5(2); the Art. 9(1) list
S22	CJEU — Case C-101/01 Lindqvist, judgment of 6 November 2003	https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:62001CJ0101	2026-08-20	Para. 47: the exception relates only to activities carried out in the course of private or family life, and internet publication to an indefinite audience is clearly not that; the result held despite a non-economic activity
S23	CJEU — Case C-212/13 Ryneš, judgment of 11 December 2014	https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:62013CJ0212	2026-08-20	Paras. 27-33: the exception must be narrowly construed, "purely" signals restrictive scope, and processing directed outwards from the private setting is not purely personal or household
S24	Commission Implementing Decision (EU) 2023/1795 of 10 July 2023	https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32023D1795	2026-08-20	The United States ensures an adequate level of protection for personal data transferred to organisations certified under the EU-U.S. Data Privacy Framework
S25	General Court — Case T-553/23 Latombe v Commission, judgment of 3 September 2025	https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:62023TJ0553	2026-08-20	The action for annulment of Decision (EU) 2023/1795 was dismissed and all pleas rejected; no appeal recorded in the consulted text

#	Title	URL	Accessed	Supports
S26	EDPB — Guidelines 07/2020 on the concepts of controller and processor in the GDPR	https://www.edpb.europa.eu/documents/guidelines/guidelines-072020-on-the-concepts-of-controller-and-processor-in-the-gdpr_en	2026-08-20	Existence, exact title and adoption date (7 July 2021) of the applicable EDPB guidance. Content not extracted — the published PDF did not parse
S27	UODO — Prezes UODO reaguje w sprawie kamer monitoringu zbierających dane z posesji sąsiadów, 8 December 2025	https://uodo.gov.pl/pl/138/3992	2026-08-20	The Polish supervisory authority's application of the narrow reading of the purely personal or household exclusion, and that it fails once processing extends beyond the private sphere
S28	MDN — Passkeys	https://developer.mozilla.org/en-US/docs/Web/Security/Authentication/Passkeys	2026-08-20	Losing an authenticator loses all the passkeys it contains; the recommendation to register multiple passkeys as backup; the private key stays in the authenticator; passkeys are scoped to a relying-party id

10. Non-legal-advice statement

This report is engineering research, not legal advice. It was produced by reading primary sources — the text of Regulation (EU) 2016/679, two CJEU judgments, one General Court judgment, one Commission implementing decision, one published decision of the Polish supervisory authority, and vendor documentation — and applying them to the verified facts of one small application. No lawyer reviewed it, and no lawyer was consulted.

Three specific cautions. First, §5a's central application — that a paid private teacher's client records most likely fall outside the household exemption — is a reasoned reading of case law that does not address these facts; it is labelled contested for that reason and it could be argued the other way. Second, the retention period the teacher actually needs is driven by Polish tax and accounting record-keeping rules, which this report deliberately did **not** research and did not estimate; the correct source is her accountant. Third, nothing here should be relied on for a decision with consequences — a dispute with a parent, a supervisory-authority enquiry, or a decision about the disclosed bank account — without professional advice. For the disclosed IBAN and tax identifier specifically, the right next step is a conversation with the named person and, at their discretion, their bank, not a code change.

11. Quick wins

Four wins, all independently shippable, none blocked by an open decision in this report. Deliberately excluded and why: removing the personal payment details from the default template is already RP-01's quick win 1 and is not duplicated here; making `Clear All Data` also clear `paymentTemplate` is blocked by RP-01 open question 4; storage error handling and the JSON backup are RP-05's quick wins 1 and 2; anything touching Firebase is blocked by the template cleanup and by a project that does not exist yet.

Rank	Quick win	Effort	Impact	Basis of ranking
1	Escape the group name in the card sink, closing the verified stored XSS	XS	High	RP-01 runtime-verified script execution from a CSV-supplied name, persisting to storage and re-firing on every load. One helper plus one interpolation. It is also the prerequisite that stops sync from turning an XSS into a credential-theft path
2	Add a committed guard that fails if IBAN-shaped or long-digit strings appear in <code>index.html</code>	XS	High	The report's central finding is a published financial identifier. A grep-based guard is the only thing that makes its removal durable against a future edit or a copy-paste. Creates one new file, touches no app code, and correctly reports a failure on today's <code>index.html</code> until the values are removed
3	Add a non-identifying placeholder hint to the group-name input	XS	Medium	Implements §6's highest-leverage minimisation measure at the point of data entry. <code>index.html</code> currently has zero <code>placeholder</code> attributes, so this is purely additive, with no logic and no storage change
4	Extend <code>.gitignore</code> to cover env files and credential JSON	XS	Medium	§3's rule made durable before any tooling arrives. The current file is six lines and covers only Node and Playwright artifacts. Prevents the most common way a secret enters a public repository

PROMPT QW-1: Escape the group name before interpolating it into the group card

Context: Repo lesson-planner, single deployed file `index.html` at the repo root, served by GitHub Pages. `App.render.groups` builds each card in a helper at `index.html:1042-1053`, which assigns `card.innerHTML` from a template literal that interpolates `${group.name}` unescaped at `index.html:1047`. RP-01 runtime-verified that a group name supplied through CSV import containing an `img` tag with an `onerror` attribute parses into a real DOM element and executes, that the payload persists to the `groupLessonPlannerData` `localStorage` key, and that it re-fires on every page load. There is no escaping helper anywhere in the file today; `App.utils` sits at `index.html:1365` onwards.

Task: Add a small pure function to `App.utils` that returns an HTML-escaped copy of a string, escaping at minimum the ampersand, less-than, greater-than, double-quote and single-quote characters, escaping the ampersand first. Apply it to the `${group.name}` interpolation at `index.html:1047` so the card renders the name as literal text.

Constraints: Do not change the rendered DOM structure, class names, element order or the surrounding text of the card, including the string that produces "{n} planned lessons" at `index.html:1049`. Do not change the three `localStorage` key names `groupLessonPlannerData`, `groupLessonPlannerSettings` and `paymentTemplate`, and do not change the persisted data shape; escaping is applied at render time only and stored values must remain byte-identical. Do not alter the sanitisation or validation of CSV import in this change. Do not switch the card to

createElement or textContent; keep the innerHTML template so the change stays a one-line diff plus a helper. No new dependencies and no build step.

Acceptance criteria: A group whose stored name is a bold-tag wrapper around the single letter x renders on the card with every angle bracket and slash visible as literal text, and produces no B element inside the card. A group whose stored name contains an img tag with an onerror attribute produces no IMG element in the document and fires no script. A group named A & B still displays as A & B. The stored value of groupLessonPlannerData is unchanged by rendering. Opening a group, editing its name and saving still round-trips the exact typed characters, including angle brackets, back into the name input.

Verification: Serve index.html locally over http://127.0.0.1, seed groupLessonPlannerData with one group whose name is an img tag carrying an onerror attribute that sets a global marker variable, reload, then assert in the console that the marker is undefined and that document.querySelectorAll('.group-card img').length is 0 and the card text contains the literal tag characters. Repeat with the name A & B and assert the card text is exactly A & B. Finally read groupLessonPlannerData back and assert the name string is unchanged.

PROMPT QW-2: Add a repository guard that fails when personal financial identifiers appear in index.html

Context: Repo lesson-planner. The default payment template at index.html:382-400 contained a real person's full name, bank IBAN, tax identification number and bank identifiers at lines 387-392, plus a personal first name at line 400, all publicly readable through view-source on the deployed GitHub Pages site and present in git history. RP-01 quick win 1 removes them from HEAD. Nothing currently prevents an equivalent value being pasted back in during a future edit, and there is no CI in the repository: the verified-facts addendum records that no .github directory exists and that every npm script named in AGENTS.md is aspirational because no root package.json is committed.

Task: Create one new executable shell script at the repository root named check-no-personal-data.sh that scans index.html and exits non-zero, printing the offending line numbers, when any of the following is found: a string matching the IBAN shape of two uppercase letters followed by six or more digits; any run of eight or more consecutive digits; or the case-insensitive fixed strings IBAN and MF0. The script must allow exactly one exception, the Ukrainian payment-purpose line, identified by content rather than by line number so the check does not break when lines move. The script must print a short explanation of what to do when it fails, and exit zero with a one-line success message otherwise.

Constraints: Do not modify index.html. Do not create a .github directory or any CI workflow; wiring the script into CI is explicitly out of scope and should be mentioned only in the script's own comment header. Do not add a package.json or any npm script. Use only POSIX shell plus grep available on macOS and Linux; do not depend on GNU-only grep flags, and where a Perl-style character class is needed, detect its availability and fall back rather than failing. Do not embed any real name, IBAN, tax identifier or bank identifier in the script, in its comments, or in its output; the script must match shapes and generic keywords only. Do not scan git history.

Acceptance criteria: Run against the index.html committed today, the script exits non-zero and reports the offending lines inside the 382 to 400 range; this failure is the expected and correct result and is the proof the guard works, because those values have not yet been removed. Run against a copy of index.html with the payment-details block and the signature line replaced by neutral placeholder text, it exits 0 and prints the success line. Run against a copy into which a string of the shape two uppercase letters followed by twenty digits has been inserted, it exits non-zero and prints that line's number. Run against a copy containing the word iban in lower case, it exits non-zero. The permitted Ukrainian payment-purpose line alone never causes a failure. The script's own source contains no digit run of eight or more characters. The script accepts an optional file path argument, defaults to index.html relative to the script's own directory, and runs correctly from any working directory.

Verification: Run the script from the repository root and again from a different directory and confirm both report the same non-zero result and the same line numbers against the

current file. Copy `index.html` to a scratch path, replace the payment-details lines and the signature line with neutral placeholder text, run the script against that copy and confirm exit 0 and the success line. Append to that cleaned copy a line containing two uppercase letters followed by twenty digits and confirm a non-zero exit naming that line. Repeat with a line containing the lower-case word `iban`. Finally delete the Ukrainian payment-purpose line from the cleaned copy and confirm the script still exits 0, proving the exception is not load-bearing for success.

PROMPT QW-3: Add a non-identifying placeholder hint to the group name input

Context: Repo `lesson-planner`, `index.html`. The group name input is a bare text input at `index.html:282` inside the group info form that starts at `index.html:279`, labelled Group Name by the label at `index.html:281`. Verified by `grep`: `index.html` contains zero placeholder attributes on any input. RP-06 identifies the free-text group name as the only field in the whole data model that can turn a schedule record into personal data about a student, and identifies adopting non-identifying group labels as the highest-leverage, zero-cost minimisation measure available. A placeholder at the point of entry is the cheapest way to make that convention visible without adding validation or copy elsewhere.

Task: Add a placeholder attribute to the group name input at `index.html:282` whose value suggests a schedule-style label rather than a person's name, for example a weekday, a time and a level. Use plain ASCII English consistent with the rest of the chrome, which `index.html:2` declares as `lang="en"`.

Constraints: Change only the markup of that one input. Do not add a `maxlength`, `required`, `pattern` or any other validation attribute; RP-01 records that no input in the file carries validation today and adding it here is a separate decision. Do not change the label text Group Name, the input id `groupNameInput`, the input type, or any surrounding element. Do not add a placeholder to any other input in this change. Do not change the three `localStorage` key names `groupLessonPlannerData`, `groupLessonPlannerSettings` and `paymentTemplate`, and do not change the persisted data shape. Do not change the blank-name fallbacks at `index.html:661` and `index.html:677`. No new dependencies and no build step.

Acceptance criteria: Opening Add Group on a fresh profile shows the hint text greyed out in the empty group name field. Typing a name replaces the hint and the typed value is what is saved. Saving with the field left empty still produces the existing untouched fallback name, unchanged from current behaviour. Opening Edit Group on an existing group shows that group's real name and no hint. The hint contains no personal name and no digits that could be mistaken for a real identifier beyond a clock time. Exactly one placeholder attribute exists in `index.html` after the change.

Verification: Serve `index.html` locally, clear `localStorage`, click Add Group and read the computed placeholder of the element with id `groupNameInput`, asserting it is non-empty. Type a name, save, reopen the group for editing and assert the input value is the typed name and that the placeholder is not displayed. Then add a second group leaving the name blank, save, and assert the card shows the same fallback name the app produced before this change. Finally `grep index.html` for placeholder and assert exactly one match.

PROMPT QW-4: Extend `.gitignore` to cover environment files and credential material

Context: Repo `lesson-planner`. The committed `.gitignore` is six lines and covers only Node and Playwright artifacts: `node_modules`, `playwright-report`, `test-results`, `e2e/.cache` and `.playwright-cache`. RP-06 establishes that a static site has nowhere to hold a secret, that Vite's own documentation says values prefixed for client exposure are embedded in the bundle at build time and must not contain sensitive information, and that GitHub secret scanning runs automatically on public repositories and reports detected partner secrets to the provider. The repository is public and is the deployment source for the live GitHub Pages site, so any credential committed here is published. No tooling that would create `env` files exists yet, which is exactly why the ignore rules should land before it does.

Task: Append a clearly commented section to `.gitignore` that ignores environment files and credential material: `.env` and every `.env.*` variant, while explicitly un-ignoring a committed

example file named `.env.example`; service-account and credential JSON files matched by the patterns `*-service-account*.json`, `service-account*.json` and `google-credentials*.json`; and `*.pem` plus `*.p12`. Keep the existing five patterns and their comment exactly as they are, and add the new rules below them under their own comment header.

Constraints: Do not create any of the ignored files, and do not create `.env.example` itself; the negation is there so a future example file can be committed deliberately. Do not remove, reorder or reword any existing line in `.gitignore`. Do not modify `index.html`, `AGENTS.md` or `LICENSE`. Do not add a `package.json`. Do not run any git history rewriting. Note that `.gitignore` only affects untracked files, so state in the added comment that it does not remove anything already committed.

Acceptance criteria: `git check-ignore -v` reports a match for each of `.env`, `.env.local`, `.env.production`, `credentials.pem`, `key.p12` and `my-service-account.json`. `git check-ignore` returns no match for `.env.example`. The five original patterns still match what they matched before: `node_modules`, `playwright-report`, `test-results`, `e2e/.cache` and `.playwright-cache`. No file other than `.gitignore` is modified, and no new file other than the changed `.gitignore` appears in `git status`.

Verification: From the repository root run `git check-ignore -v` on each of the six paths listed above and confirm every one reports a matching rule and line. Run `git check-ignore -v .env.example` and confirm it reports no match, exiting non-zero. Run `git check-ignore -v` on `node_modules` and `.playwright-cache` and confirm the original rules still match. Finally run `git status` and confirm `.gitignore` is the only modified path.